

Platform Engineering & DevOps

Chaos Engineering Release the Experiment!

Karen Quijada



DEVOPSDAYS.PE



27 - 28 AGO 2026



Karen Quijada

Regional Solution Architect DevOps & AI · Optima Latam/DevOpsDays Santiago

+ de 20 años de experiencia en Tecnología, Speaker Internacional y en apoyo activo a comunidades en Chile. Ingrese al mundo de DevOps hace aprox. 8 años, MIT Certificate. Co-organizadora de DevOpsDays Santiago de Chile.

SDLC/ADLC

AI

VSM

CULTURA

Chaos Engineering Release the Experiment!

Construyendo sistemas que sabemos que van a fallar



Todo estaba verde...

CPU

✓ OK

Memory

✓ OK

API

✓ OK

Database

✓ OK

Kubernetes

✓ OK

Monitorear que algo funciona no es lo mismo que saber cómo reaccionará cuando falle.

Los dashboards verdes generan confianza. El problema es que esa confianza a veces no está respaldada por evidencia real. **Nunca habíamos probado qué ocurría cuando algo fallaba.**



Chaos Engineering no es romper cosas

CHAOS

- **Fallas aleatorias sin plan**
- **Romper producción a ver qué pasa**
- **Sin hipótesis previa**
- **Sin límites definidos**
- **Sin aprendizaje sistemático**

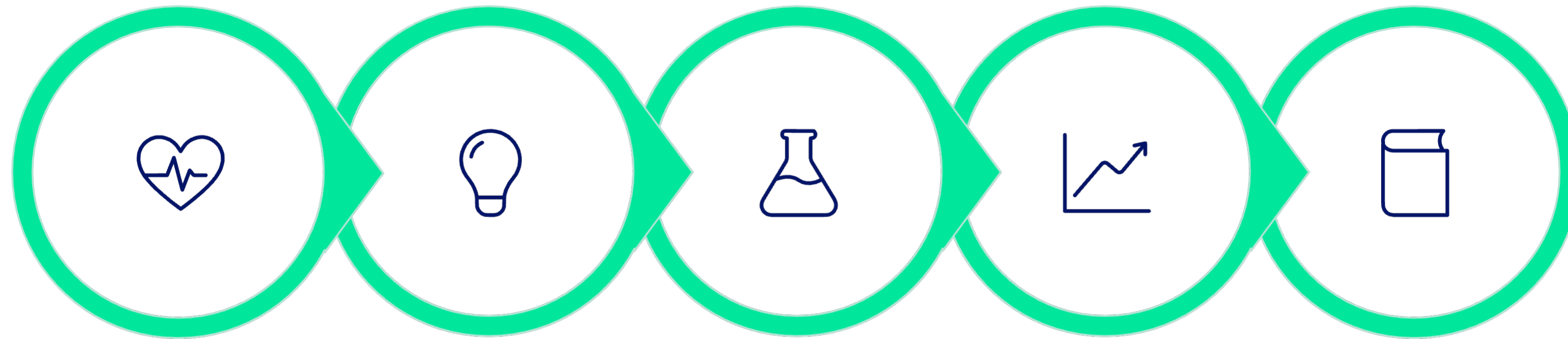
CHAOS ENGINEERING

- Hipótesis clara antes del experimento
- Experimentos controlados y reproducibles
- Blast radius definido y acotado
- Observabilidad en cada paso
- Aprendizaje documentado y accionable



Chaos Engineering no busca generar incidentes. Busca descubrirlos antes.

El método: del estado estable al aprendizaje



**Estado
estable**

Hipótesis

Experimento

Observar

Aprender

Hipótesis de ejemplo

"Si esta instancia desaparece, el usuario no debería notarlo."

Experimento

Eliminar una instancia en staging y medir la experiencia del usuario.

Resultado

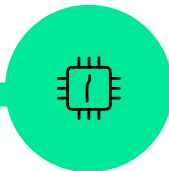
¿La hipótesis sobrevivió? Eso es lo que necesitamos saber.

¿Qué podemos romper?



Nivel 1 – Proceso / Pod / Container

El punto de partida. Mata un pod, elimina un proceso.



Nivel 2 – CPU / Memory

Simula presión de recursos. ¿Tu app degrada con gracia?



Nivel 3 – Network / Latency / Packet Loss

La red siempre miente. Ponla a prueba.



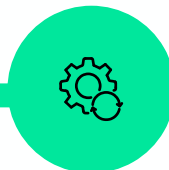
Nivel 4 – DNS / Dependencias externas

¿Qué pasa si tu DNS falla? ¿O un proveedor externo?



Nivel 5 – Databases / External APIs

Las dependencias críticas merecen escenarios de falla reales.



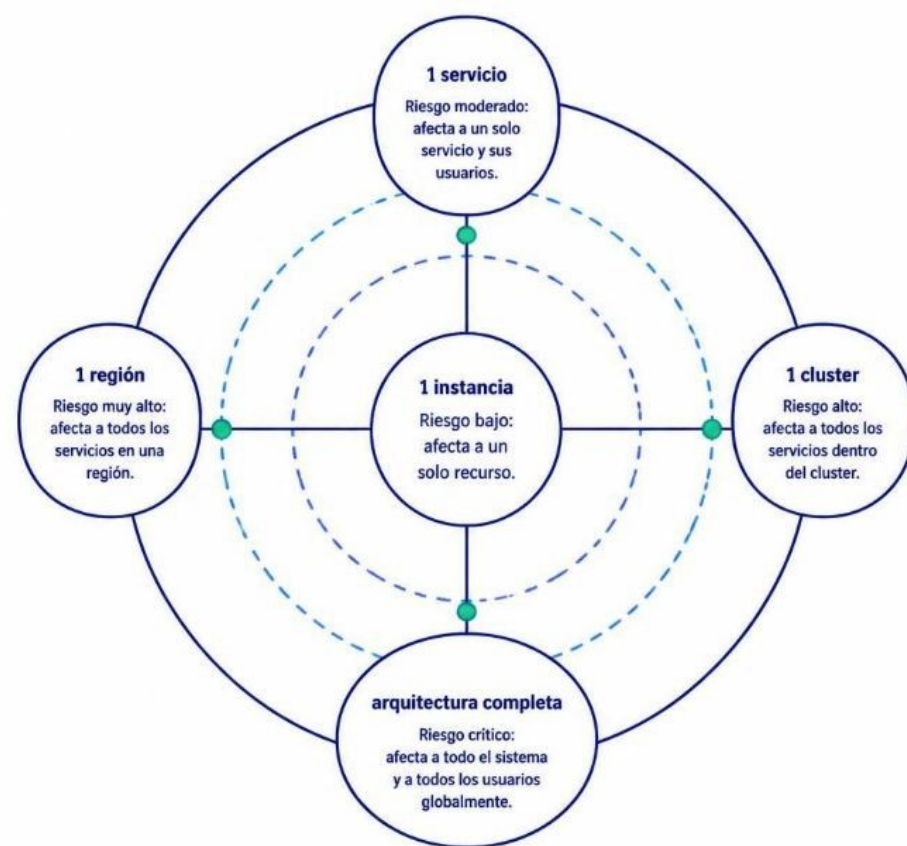
Nivel 6 – Flujos completos de negocio

El nivel final: ¿el negocio sobrevive un fallo en cascada?

Empieza pequeño. Aprende. Aumenta la dificultad.

CHAOS ENGINEERING

Blast Radius: ¿cuánto puedes controlar?



¿Por qué importa?

Definir el blast radius antes del experimento es lo que separa a un ingeniero de Chaos Engineering de alguien que simplemente... apaga servidores en viernes.

"No estoy preparado para hacer Chaos Engineering"

Buena noticia: no necesitas estarlo. El Blast Radius no es solo un concepto de seguridad — es tu nivel de preparación hecho explícito. No necesitas buy-in organizacional, un runbook aprobado por tres comités, ni un equipo de SRE para empezar.

● Hoy mismo

Mata un proceso en tu laptop o en staging. Blast radius: exactamente tú.

● Esta semana

Un pod en un namespace de staging. Blast radius: tu equipo lo nota, producción no.

● cuando estés listo

Un servicio real, con alertas activas y un rollback plan. Blast radius controlado.

El blast radius más pequeño posible es... tu laptop. Empieza ahí.
El tamaño de tu primer experimento no define tu ambición — define tu disciplina.

Sin observabilidad, solo es caos

La pregunta que importa

¿Qué vio el usuario?

Introducir una falla sin poder observarla es como operar a ciegas. La observabilidad no es opcional en Chaos Engineering — es el fundamento.

Necesitas los cuatro pilares

- **Metrics** — el pulso del sistema
- **Logs** — la historia de lo que pasó
- **Traces** — el camino de cada request
- **SLOs** — el contrato con el usuario

Usuario
¿Experimentó degradación o error?

Aplicación
¿Activó el circuit breaker? ¿Retries?

Servicios
¿Dónde se propagó la falla?

Datos
¿Consistencia garantizada bajo estrés?



Enough slides.

**Let's break
something.**

DEMO EN VIVO

Pero algo está cambiando...

La superficie de fallas no deja de crecer. Cada evolución arquitectónica introduce nuevos modos de fallo que no existían antes. Lo que funcionaba para monitorear un monolito no alcanza para una arquitectura de agentes.

→ Traditional Application

Fallas predecibles, bien documentadas

→ Microservices

Fallas en cascada, latencia distribuida

→ Cloud Native

Efímero, auto-escalable, no determinístico

→ Distributed Systems

CAP theorem en producción todos los días

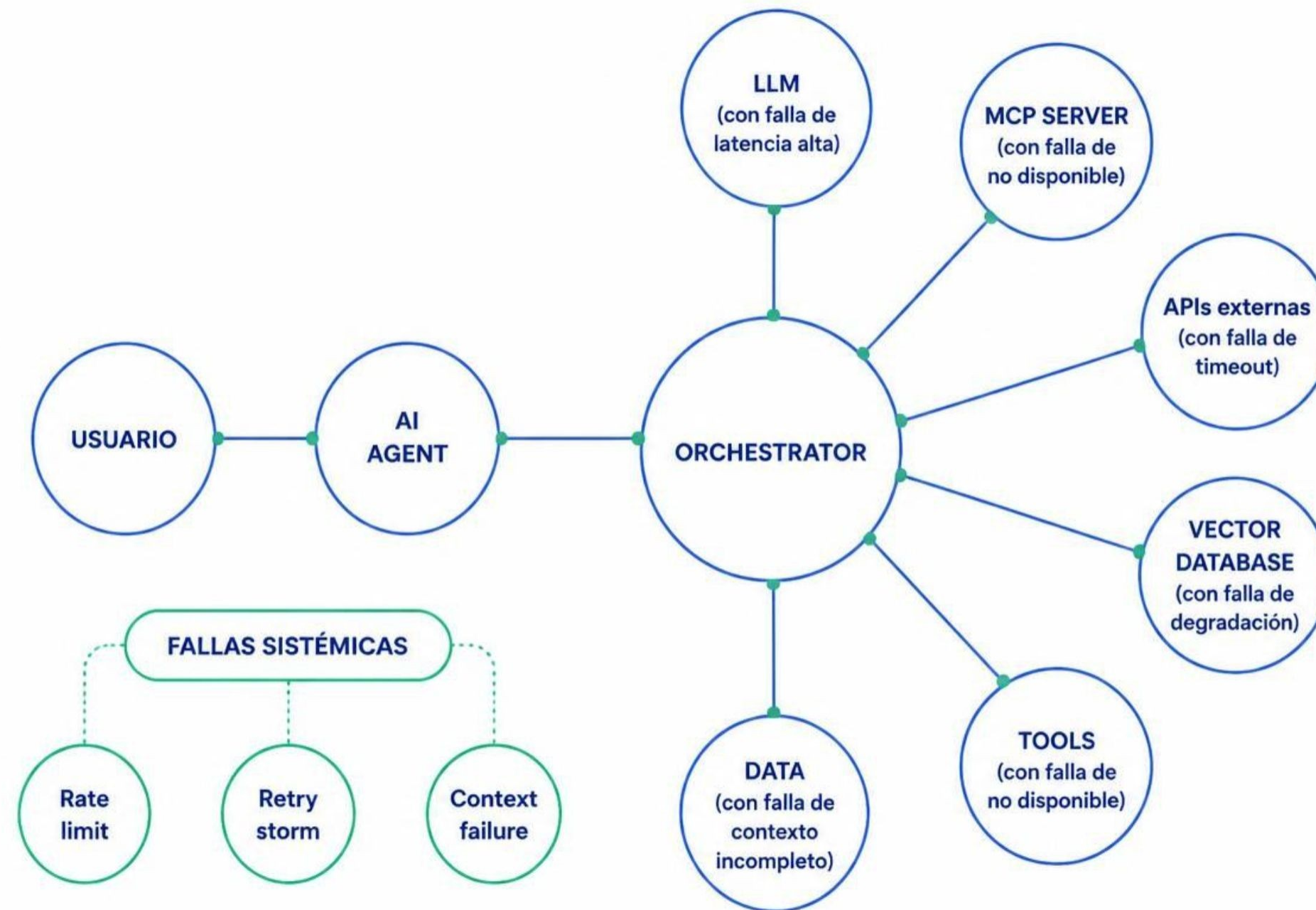
→ AI + Agents

Fallas probabilísticas, contextuales, emergentes



❏ Nuestra superficie de fallas está creciendo. ¿Nuestras prácticas de resiliencia también?

¿Qué pasa cuando falla un agente?



¿Nuestro sistema sigue siendo resiliente cuando el LLM tarda 30 segundos? ¿Cuando el MCP server no responde? ¿Cuando el agente entra en un retry loop?

Chaos Engineering para sistemas agénticos

TRADITIONAL CHAOS

CPU stress

Memory pressure

Network latency

Container kill

Kubernetes chaos

Database failure

AGENTIC CHAOS

LLM latency

Model unavailable

MCP failure

Tool unavailable

Rate limits

Context
degradation

Agent retry loops

Dependency
failures

Chaos Engineering ya no termina en Kubernetes.

Empieza el lunes

01

Elige un servicio

Uno que conozcas bien. Que sea crítico pero acotado.

02

Define su steady state

¿Qué significa "funciona"? P95 latency, error rate, throughput.

03

Escribe una hipótesis

"Si X falla, el usuario no debería notarlo porque Y."

04

Introduce una falla pequeña

Blast radius mínimo. Staging primero. Observabilidad encendida.

05

Observa y aprende

¿Sobrevivió tu hipótesis? Documenta. Mejora. Repite.

La resiliencia no significa que tu sistema nunca falle.

Significa saber qué ocurrirá cuando falle.

Break things. Learn things. Build better systems.



Gracias

Conectemos

Información de contacto

Email: kquijada@optimalatam.com / karen.quijada@devopsdayschile.cl

Instagram: [@historiasCLCafe](https://www.instagram.com/historiasCLCafe)

Linkedin: [/in/karenquijadazuniga](https://www.linkedin.com/in/karenquijadazuniga)

